

Reinforcement Learning I – How Agents Learn by Trial and Error

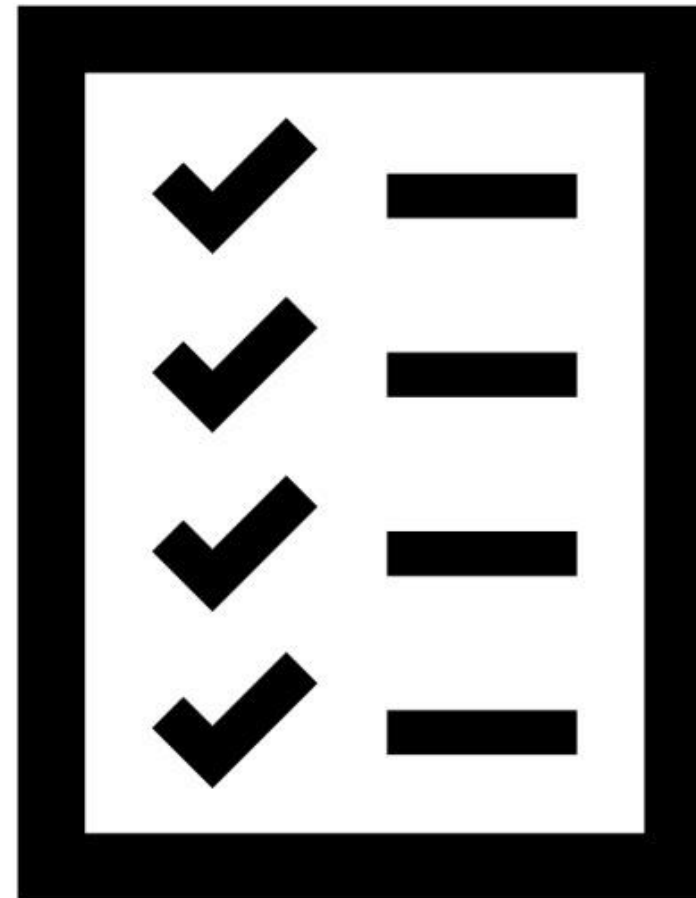
David Goll
Jill Barvencik

**Design IT.
Create Knowledge.**

www.hpi.de



Attendance List





<https://hpi.de/ki-servicezentrum/>

Overview

Gefördert durch:



Bundesministerium
für Forschung, Technologie
und Raumfahrt

Four AI Service Centres in Germany

KI Service
Zentrum
by Hasso-Plattner-Institut

HPI



KI-Servicezentren

Goal:
Reduce barriers to the implementation
of AI applications in society and the
economy

AI Service Centre Berlin-Brandenburg

Structure



Gefördert durch:



Bundesministerium
für Forschung, Technologie
und Raumfahrt



Newsletter

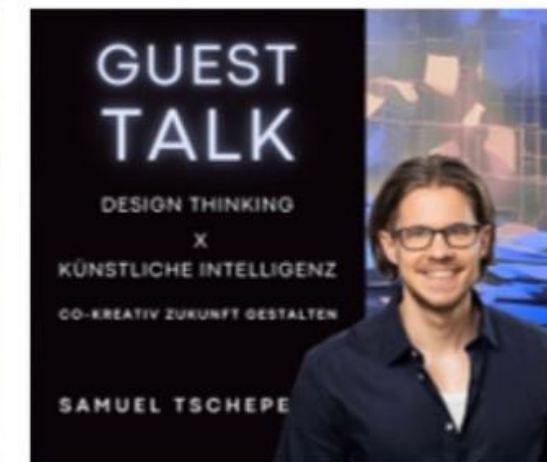
Education

Talks



tele-task.de/series/1463

- Guest talks about research and innovation

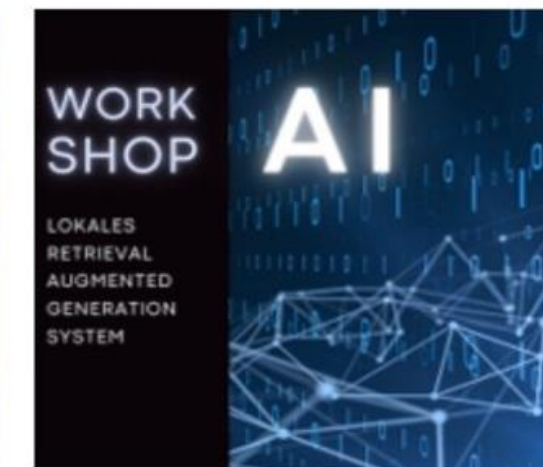


Workshops



aimaker.community

- Practical topics
- Example topics: Speech2summary, Docker for ML, semantic search



MOOCs



open.hpi.de/channels/ai-service-center

- ChatGPT: What does generative AI mean for our society?
- Profitable AI
- Understanding and avoiding AI biases



Gefördert durch:



Bundesministerium
für Forschung, Technologie
und Raumfahrt



Sprechstunde buchen

Consulting

AI consultation hours

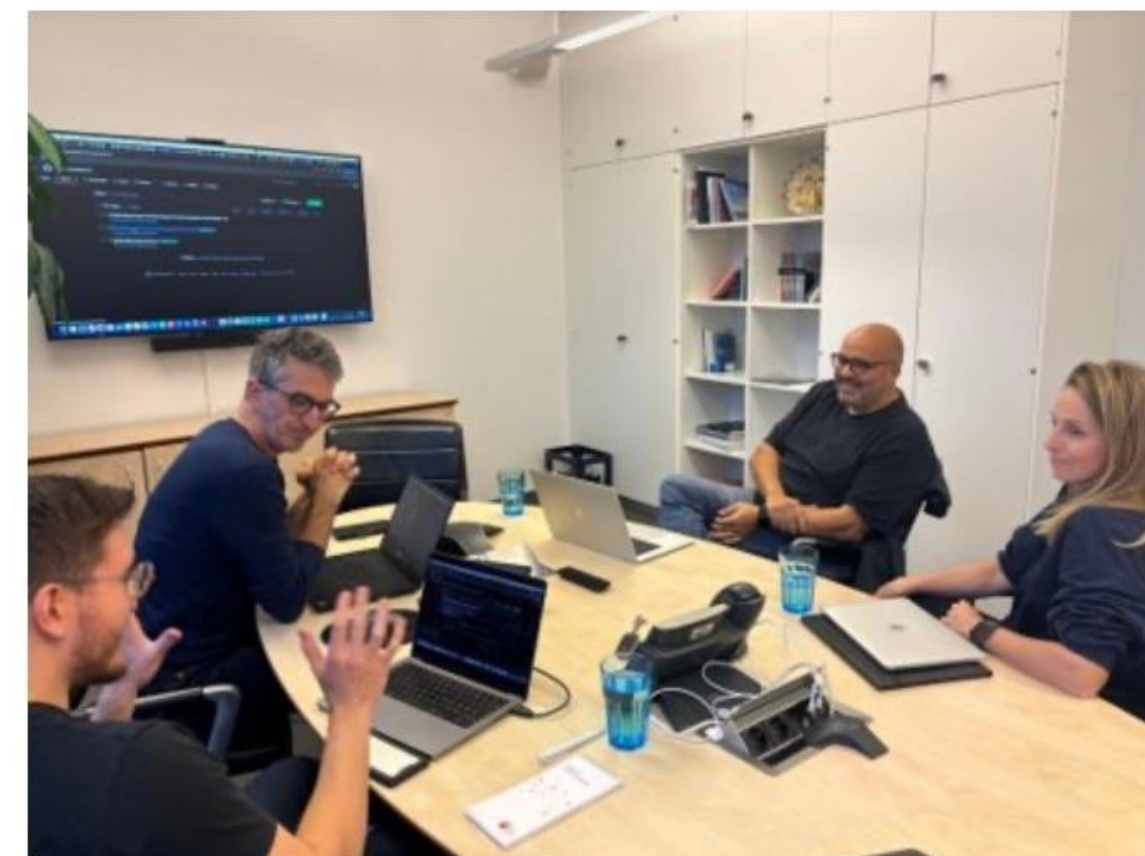
- Answering questions:
 - about AI infrastructure
 - about AI models & frameworks
 - about AI use cases

AI pilot projects

- Co-development of a prototype
- Application every three months
- Selection criteria, e.g. AI maturity, public benefit
- Publication of results

Cooperation

- Jointly organized network meetings



Previous AI pilot projects

- Generation of math problems
- Generation of easy language
- Generation of upcycling suggestions
- Reduction of food waste
- Dating by handwriting



Jetzt bewerben!

KI-Prototyp Leichte-Sprache-Assistent (LeISA)

Vereinfache Texte mit LLMs! - Zum Projekt auf GitHub

Originaltext

Das KI Servicezentrum Berlin-Brandenburg ist ein Projekt vom Hasso-Plattner-Institut. Unser Ziel ist, den Einsatz von Künstlicher Intelligenz in Wirtschaft und Gesellschaft zu fördern.

Unser Schwerpunkt liegt auf der Unterstützung von Start-ups, kleinen und mittelständischen Unternehmen, öffentlichen Einrichtungen, Studierenden und der Forschung. Wir bieten kostenlose KI-Konsultationen, Schulungen und Workshops. Zudem entwickeln wir KI-Anwendungen, die speziell auf Ihre Bedürfnisse zugeschnitten sind.

Leichte Sprache

„Unser Ziel“
Wir möchten, dass viele Menschen und Unternehmen KI-Anwendungen nutzen können.

„Wer kann sich an uns wenden?“
Studierende
Kleine und mittelständische Unternehmen
Start-ups
Öffentliche Einrichtungen

„Was bieten wir an?“
Kostenlose Beratung
Bildungsmaterialien
Infrastruktur, um KI-Anwendungen zu entwickeln

„Beispiele für unsere Arbeit“
Ein Unternehmen möchte eine App entwickeln, die Menschen bei der Einkaufsliste hilft.
Eine Forschungsgruppe möchte ein System bauen, das Menschen bei der Krankenvorgang unterstützt.
Wir helfen Ihnen gerne!

Hinweis zur Übersetzung: Dieser Text wurde mit einer KI erstellt.
Bitte verarbeiten Sie keine personenbezogenen Daten mit dem Tool. Falls der Prototyp einmal nicht verfügbar sein sollte, kontaktieren Sie uns gern, wir kümmern uns darum.

Vereinfachen | Löschen

Das KI-Prototypen-Tool ist ein gemeinsames Projekt der Brandenburgischen Landesregierung und des Hasso-Plattner-Instituts. Es wird durch das Ministerium für Wirtschaft, Arbeit und Energie der Brandenburgischen Landesregierung gefördert. Die Entwicklung des Tools wird durch das Hasso-Plattner-Institut unterstützt.

Gefördert durch:



Bundesministerium
für Forschung, Technologie
und Raumfahrt

github.com/aihpi/leichte-sprache



Zugangsanfrage
aisc.hpi.de

Infrastructure

Gefördert durch:



Bundesministerium
für Forschung, Technologie
und Raumfahrt



- **Free access**
- No production operation
 - Data should be **anonymised** or **synthesised**
 - No **hosting** of products
- **Reporting & publication** by users
- **Old rights** remain with users
- **New rights** remain with users
 - Granting of rights of use for research and teaching

Training

- 64 NVIDIA H100 GPU

Inference

- 40 NVIDIA A30 GPU

ARM Server

- Ampere Altra Max M128-30 CPU
- 2 x NVIDIA L40 GPUs

GPU Server

- AMD Epyc CPU
- 8 x NVIDIA L40S GPU

Edge

- ARMv8 CPU
- NVIDIA Jetson AGX Module

Neuromorph

- 288 SpiNNaker2 Chips

Storage

- 1.5 PB NVMe

Network

- 400 Gb/s Infiniband
- 200 Gb/s Ethernet

Personal Introduction – David Goll

Background

Physics

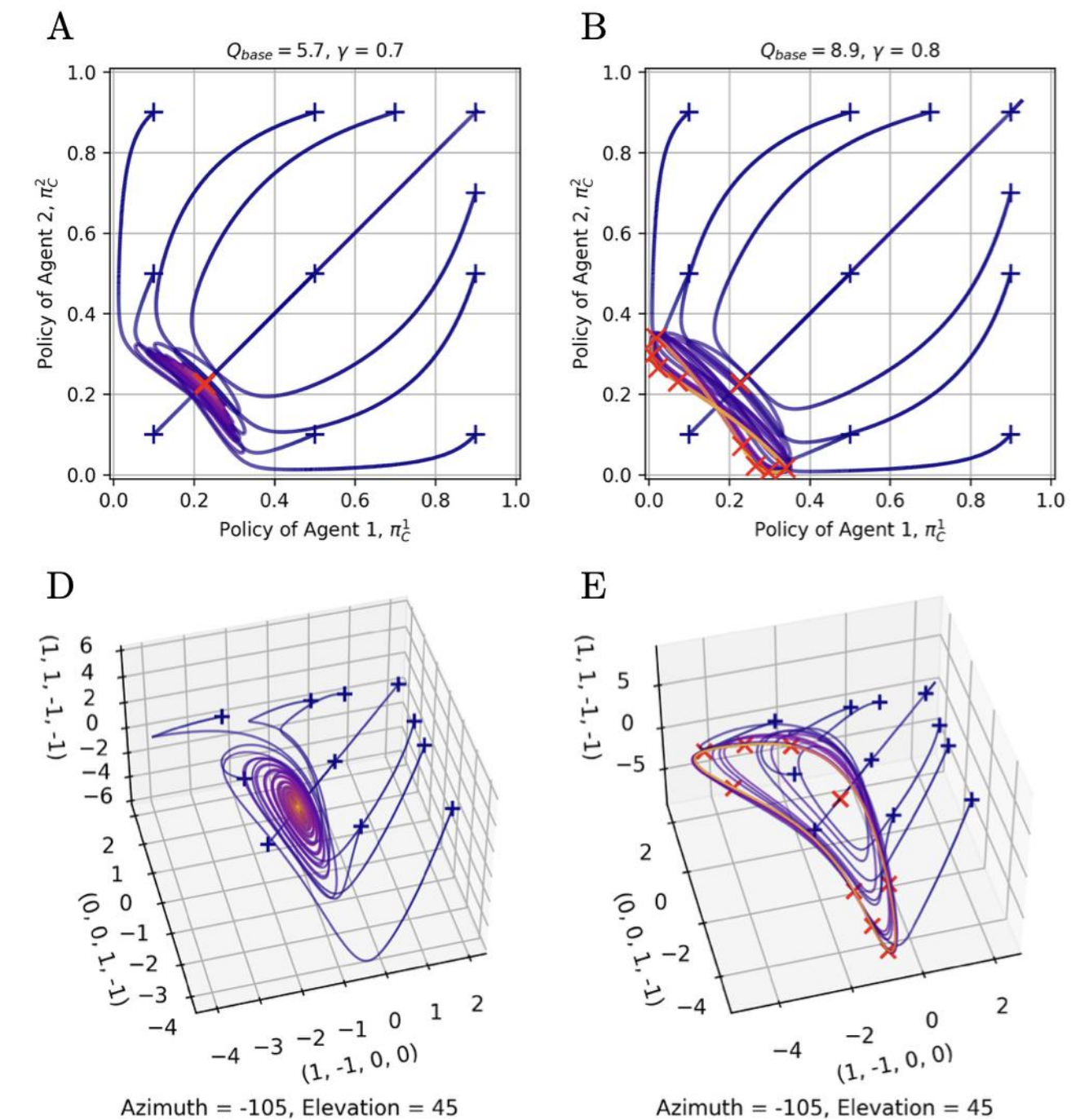
Master's thesis

“Analysis of Multi-Agent Reinforcement Learning from a Statistical Physics Perspective”

<https://doi.org/10.3390/app16073524>

Main message

Complex dynamics can arise even in simple MARL systems



Personal Introduction – Jill Barvencik

Background

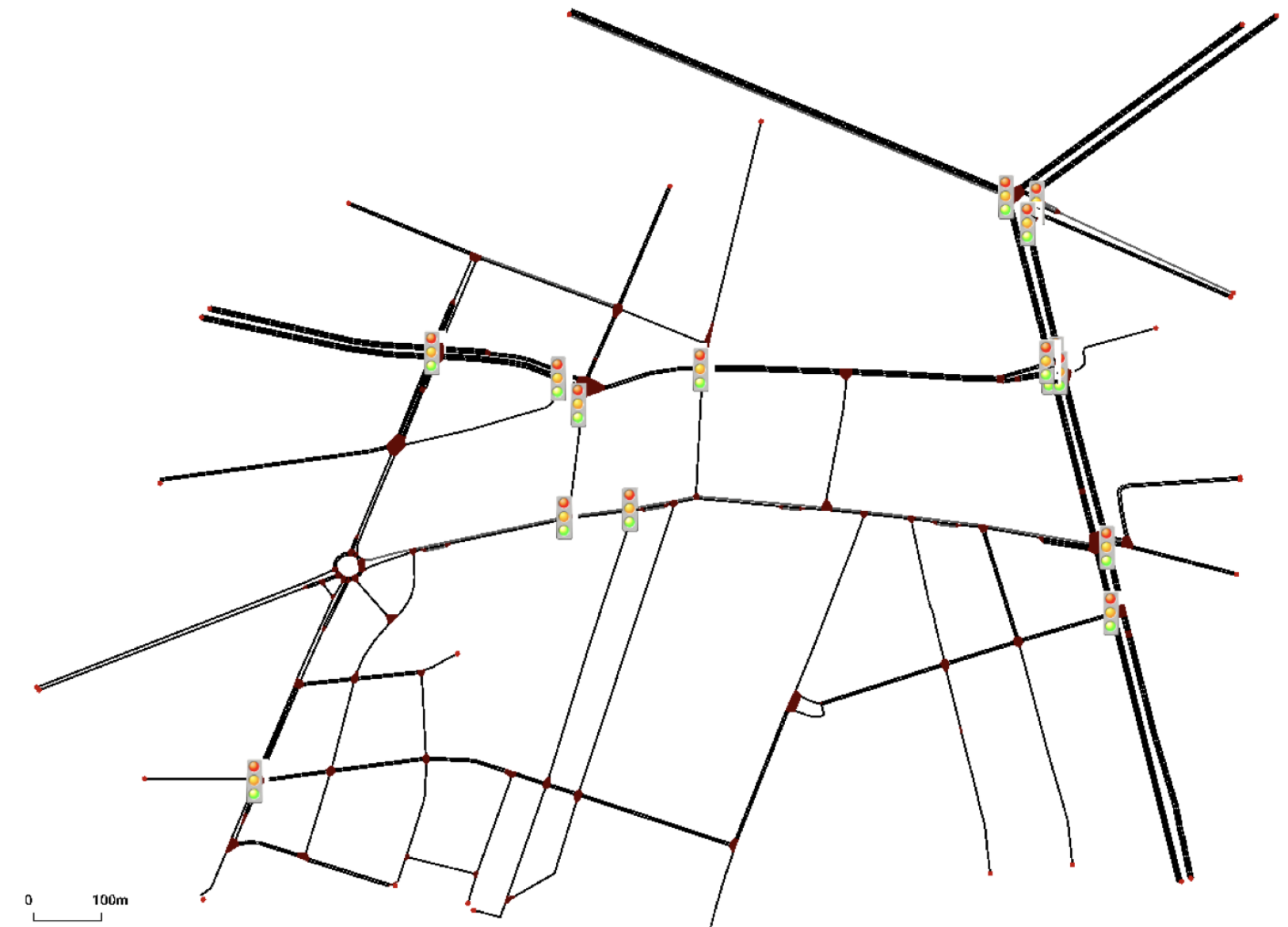
Mechanical Engineering and Computational Engineering Science

Master's thesis

“Deep Multi-Agent Reinforcement Learning for Real-World Large-Scale Traffic”

Main message

Used RL to stabilise seq2seq training and improve robustness beyond sparse reward signals



Personal Introduction – Your Turn

Please introduce yourself in 1–2 sentences

Two questions to get us started:

What is your background?

Why are you interested in RL?

Reinforcement Learning I – How Agents Learn by Trial and Error

David Goll
Jill Barvencik

**Design IT.
Create Knowledge.**

www.hpi.de



Agenda

What we will cover today

1 Introduction — core concepts, historical overview

2 Demonstrator — RL Lab

Break

3 Interactive — Installation & Exploration of RL Lab

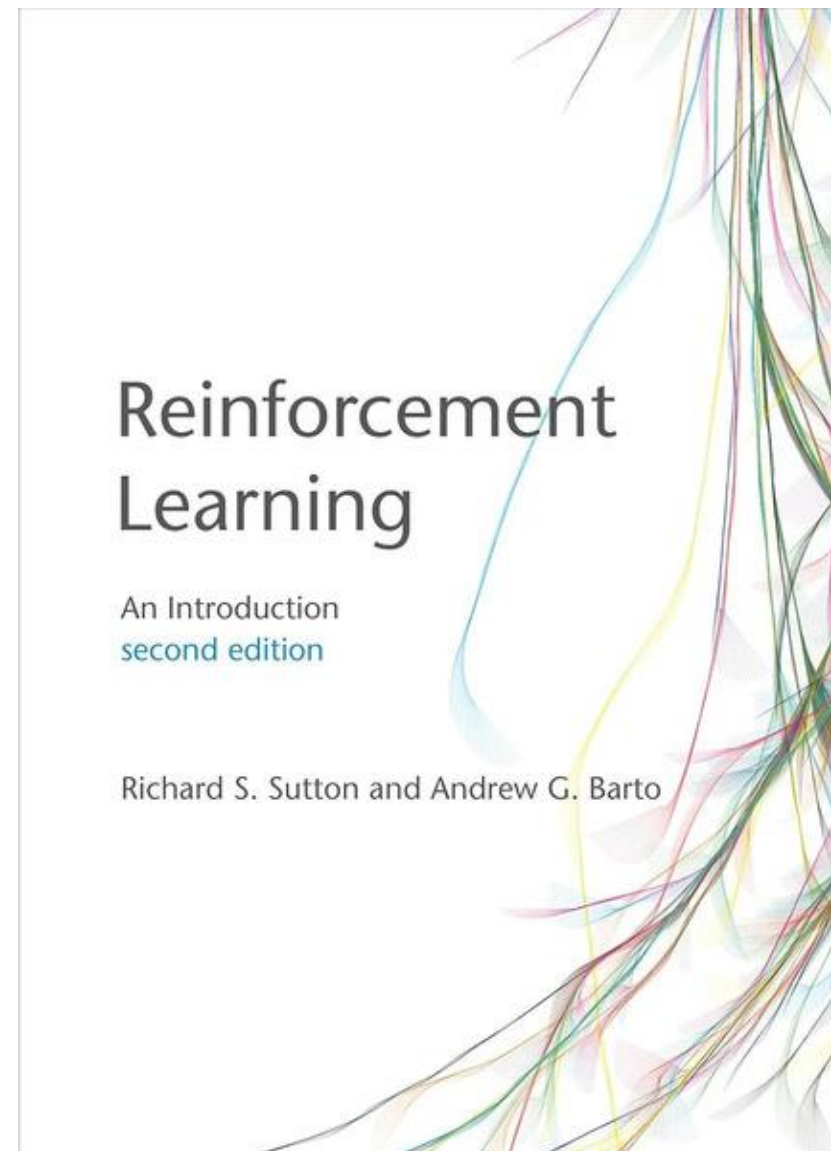
4 Today and Beyond — Modern Applications of RL

5 Interactive — Pen and Paper Exercise

Motivation and Context I

“Of all the forms of machine learning, reinforcement learning is the closest to the kind of learning that humans and other animals do.” – Sutton & Barto

kisz@hpi.de
hpi.de/kisz



Motivation and Context II

“Of all the forms of machine learning, reinforcement learning is the closest to the kind of learning that humans and other animals do.” – R. S. Sutton

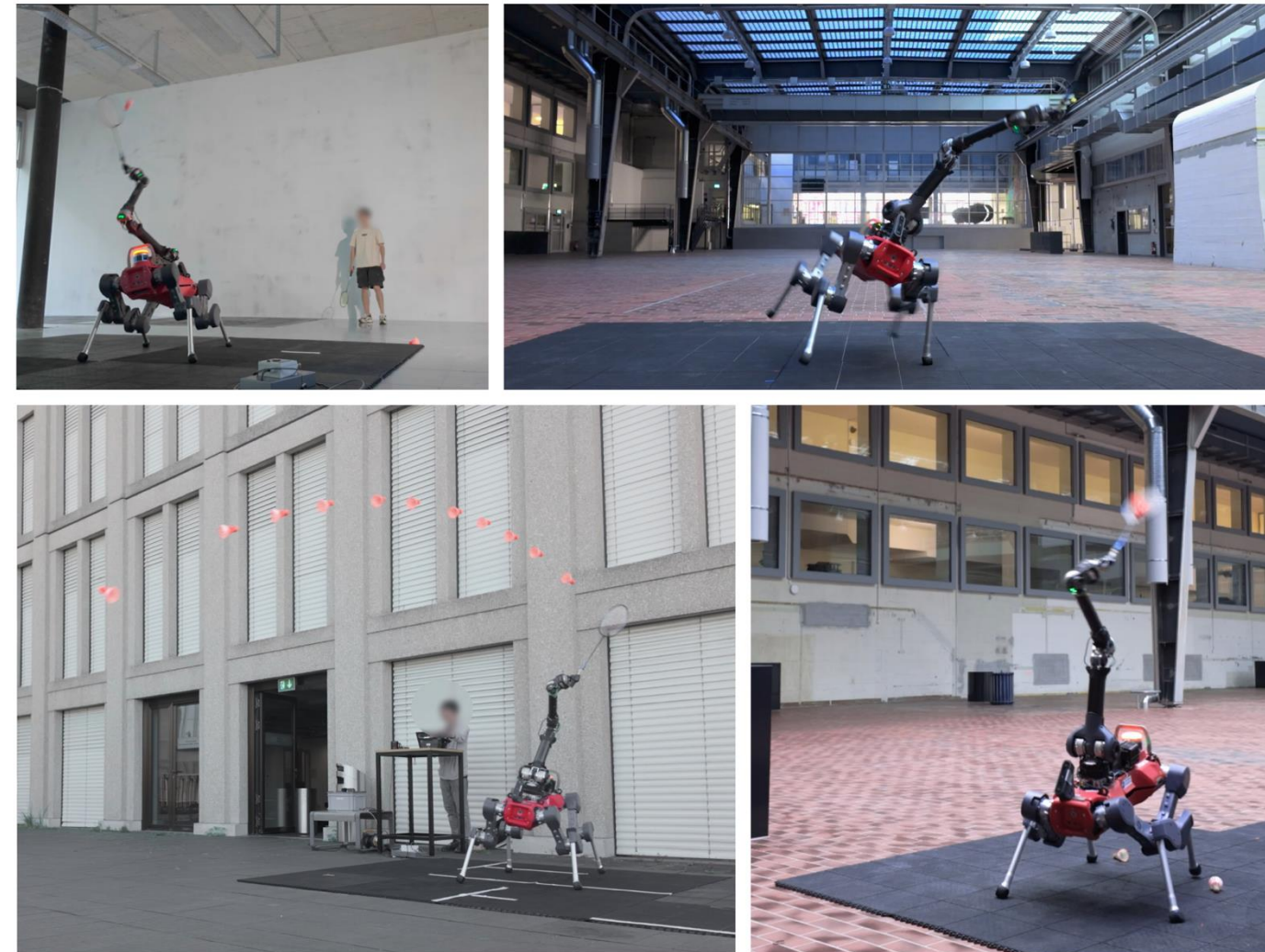


Fig. 1. Deployment of the badminton control policy on our legged manipulator. Our system operates entirely on onboard perception and computation for shuttlecock detection, trajectory prediction, and limb control. It has been tested in various environments, including the lab, a historic machine hall, and outdoor settings.

Video: ETH Zürich, <https://www.youtube.com/watch?v=2hq-rWSowdo>

Original paper: Ma et al., April 2025, <https://arxiv.org/pdf/2505.22974v2>

Motivation and Context III

Where reinforcement learning sits among the ML paradigms

Supervised Learning

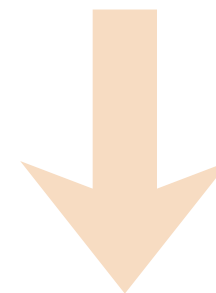
- Models are **trained on labeled data** to predict outputs

Reinforcement Learning

- Learning optimal behavior by **interacting with an environment**

Unsupervised Learning

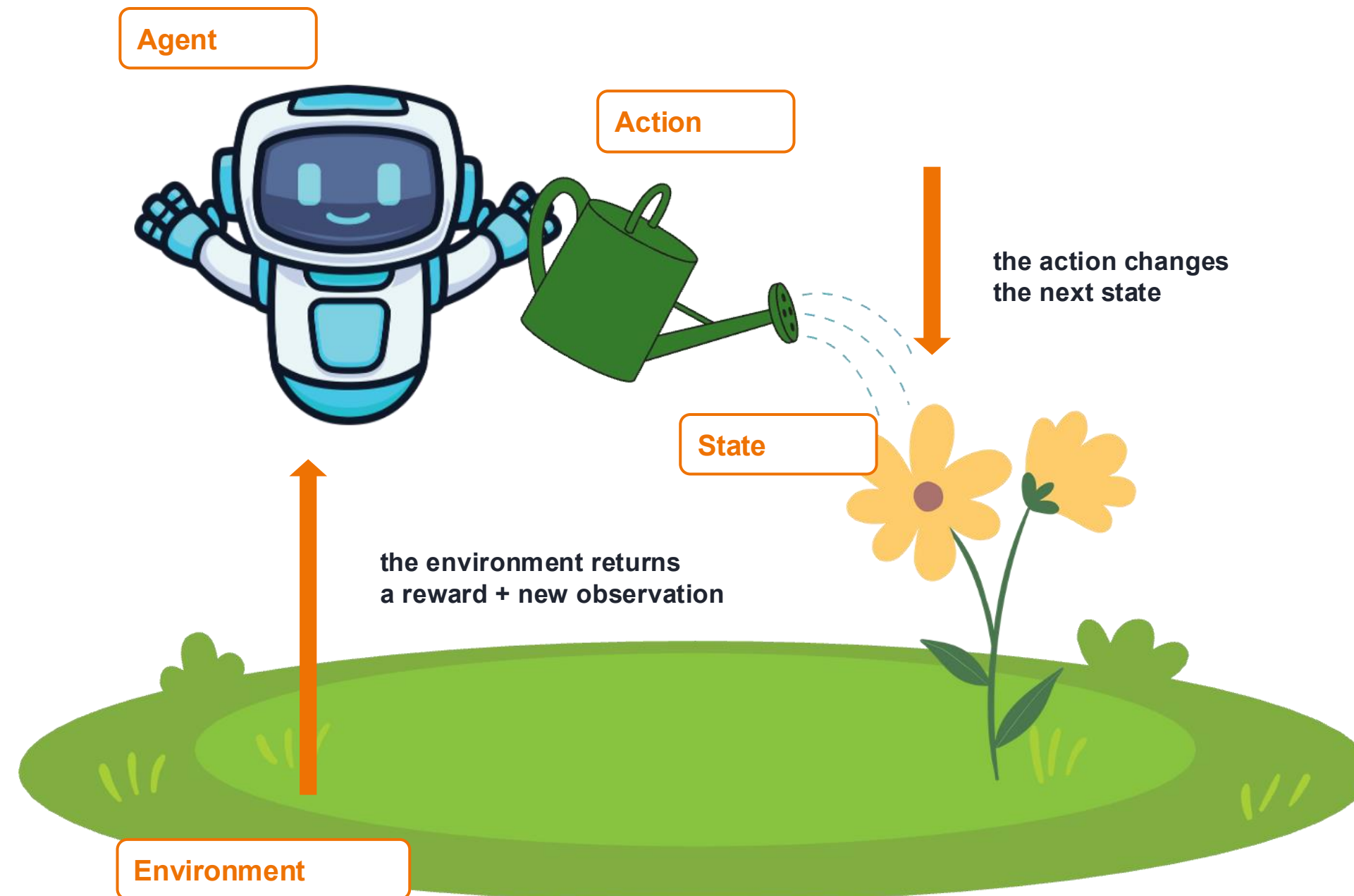
- Learn to **identify previously unknown patterns** and relationships in unlabeled data



used when **optimal actions are too complex or unknown to define** but can be learned through experience

Key RL Terms

How an agent learns by interacting with its world



- **Agent**
the decision maker
- **Environment**
the world it interacts with
- **State**
the current situation
- **Action**
what the agent can do
- **Reward**
feedback from the environment
- **Policy**
how the agent chooses actions

RL Lab demo: Untrained Agent

Home

RL Lab

Interactive Reinforcement Learning Visualization

KI Service Zentrum
by Hasso-Plattner-Institut

HPI

Geländer-Lunch
Forschungszentrum
für Technologie,
Technik und
Innovationen

Configuration

Environment

FrozenLake-v1-NoSlip

Select environment

Algorithm

Q-Learning

Select algorithm

Random Seed

518337443

Same seed = same run. Click the dice to draw a new random seed.

START TRAINING

PLAY POLICY

EVALUATE POLICY

Learning Parameters

Num Episodes

500

Training episodes. Must be an integer.

Discount Factor

0.95

$0 \leq \gamma < 1$ - importance of future rewards

Exploration Rate

0.1

$0 \leq \epsilon \leq 1$ - probability of random action

Learning Rate

0.1

$0 < \alpha \leq 1$ - controls how much new information overrides old

Q-Value Initialization

Strategy

Fixed

Q-Base Value

0

Fixed value for Q-value initialization

Environment

Ready to train a policy

About FrozenLake (No Slip)

About Q-Learning

Policy: Q-Table

Min Q: 0.000

Max Q: 0.000

Avg Q: 0.000

0	0.00	1	0.00	2	0.00	3	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00		0.00		0.00		0.00	
4	0.00	5	0.00	6	0.00	7	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00		0.00		0.00		0.00	
8	0.00	9	0.00	10	0.00	11	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00		0.00		0.00		0.00	
12	0.00	13	0.00	14	0.00	15	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00		0.00		0.00		0.00	

Lowest (global)

Highest (global)

Arrow colors show Q-values across the entire table (violet = global min, orange = global max). Cyan borders highlight the greedy action (highest Q-value) for each state (not shown if there's a tie). Grey tiles are terminal states (holes/goal): their values are never used or updated, so they keep whatever the initialization gave them and are excluded from the statistics.

Training Progress

Episodes Trained:

0

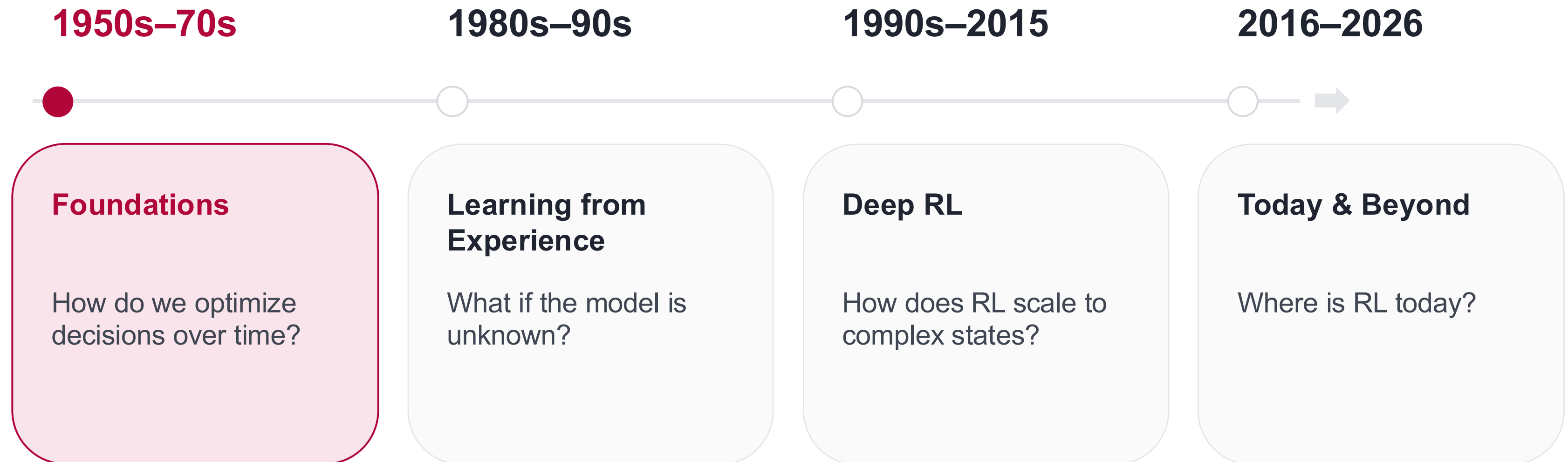
No training data yet

Average return will appear as training progresses

Live charts

How Did We Get Here?

Four questions that moved reinforcement learning forward



1950s–70s

How Do We Optimize Decisions Over Time?

MDP · Bellman · Dynamic Programming

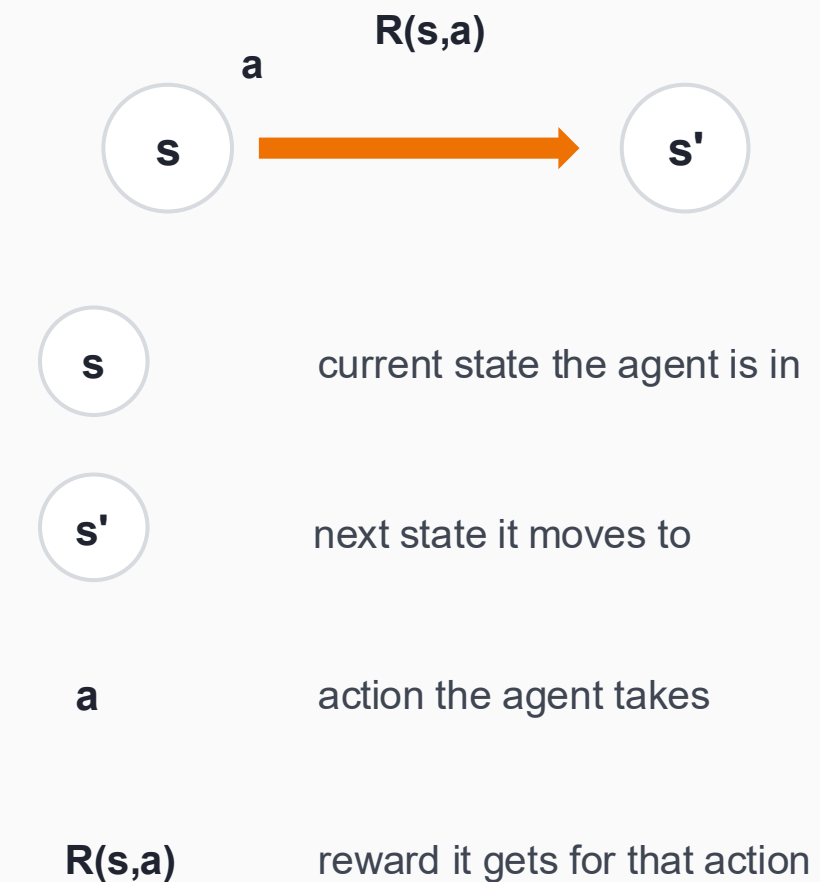
**1950s–70s
Foundations**
MDP, Bellman, DP

1980s–90s
Learning from Experience
TD and Q Learning

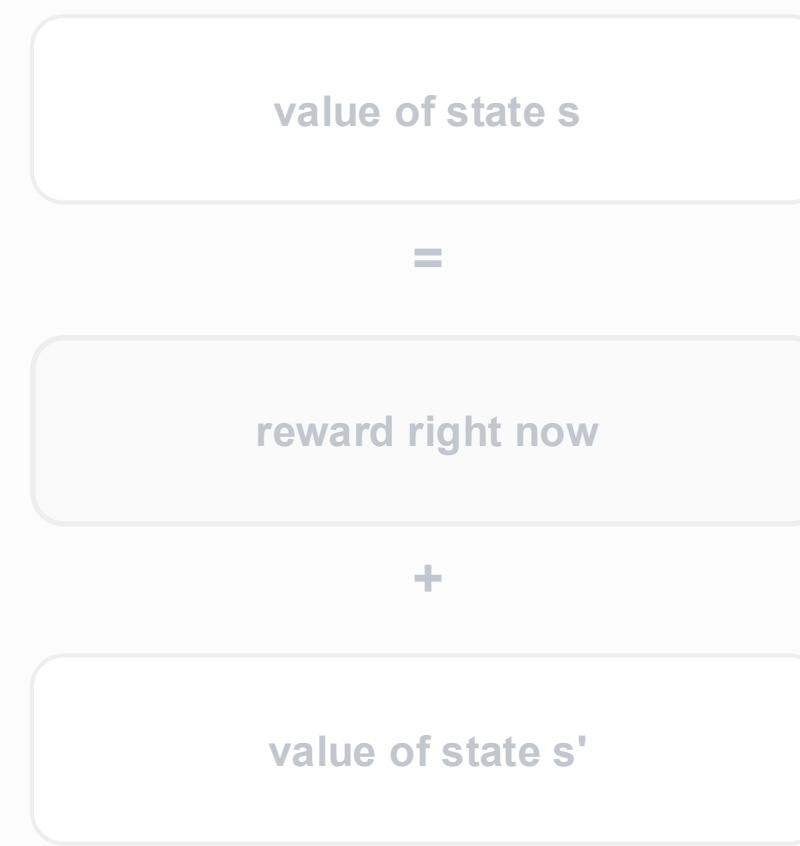
1990s–2015
Deep RL
Networks estimate values

2016–2026
Today & Beyond
RL in the real world

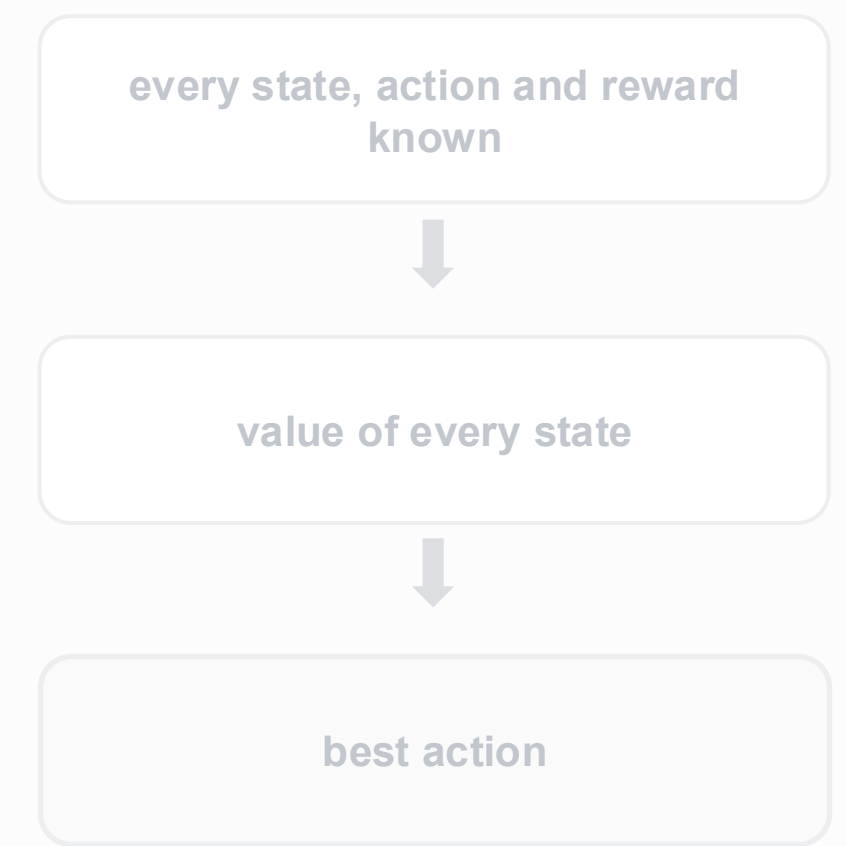
MDP: a mathematical description of the world as a decision problem



Bellman equation: links the value of today to reward now and future value



Dynamic Programming: computes the best action by sweeping all states, if the whole world is known



1950s–70s

How Do We Optimize Decisions Over Time?

MDP · Bellman · Dynamic Programming

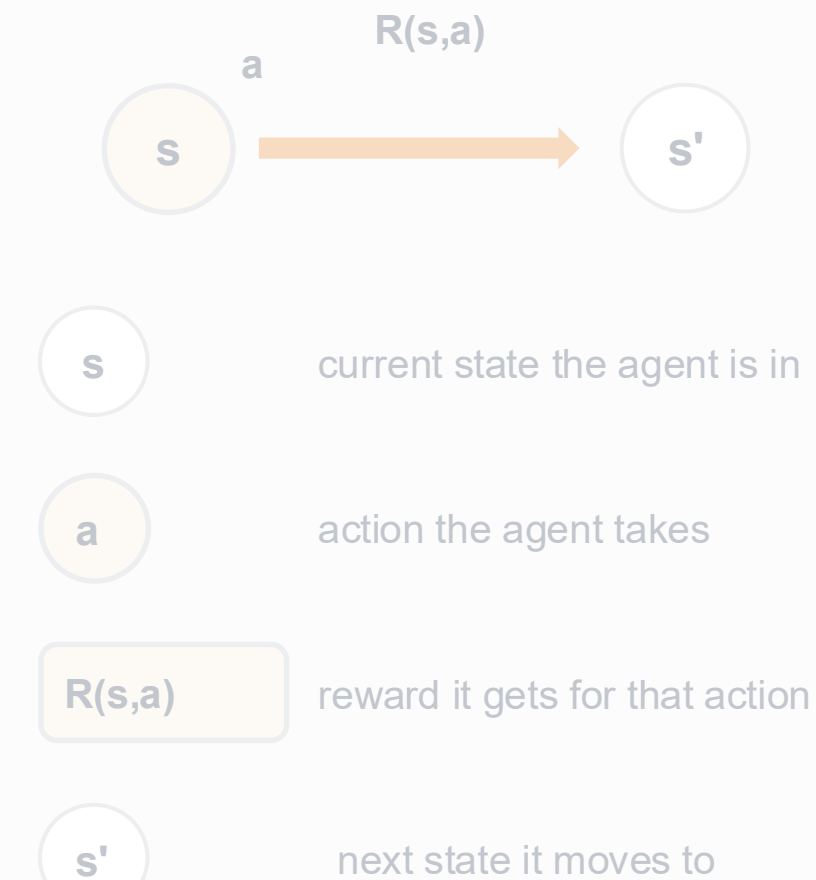
**1950s–70s
Foundations**
MDP, Bellman, DP

1980s–90s
Learning from Experience
TD and Q Learning

1990s–2015
Deep RL
Networks estimate values

2016–2026
Today & Beyond
RL in the real world

MDP: a mathematical description of the world as a decision problem



Bellman equation: links the value of today to reward now and future value

value of state **s**

=

reward right now

+

value of state **s'**

Dynamic Programming: computes the best action by sweeping all states, if the whole world is known

every state, action and reward known



value of every state



best action

1950s–70s

How Do We Optimize Decisions Over Time?

MDP · Bellman · Dynamic Programming

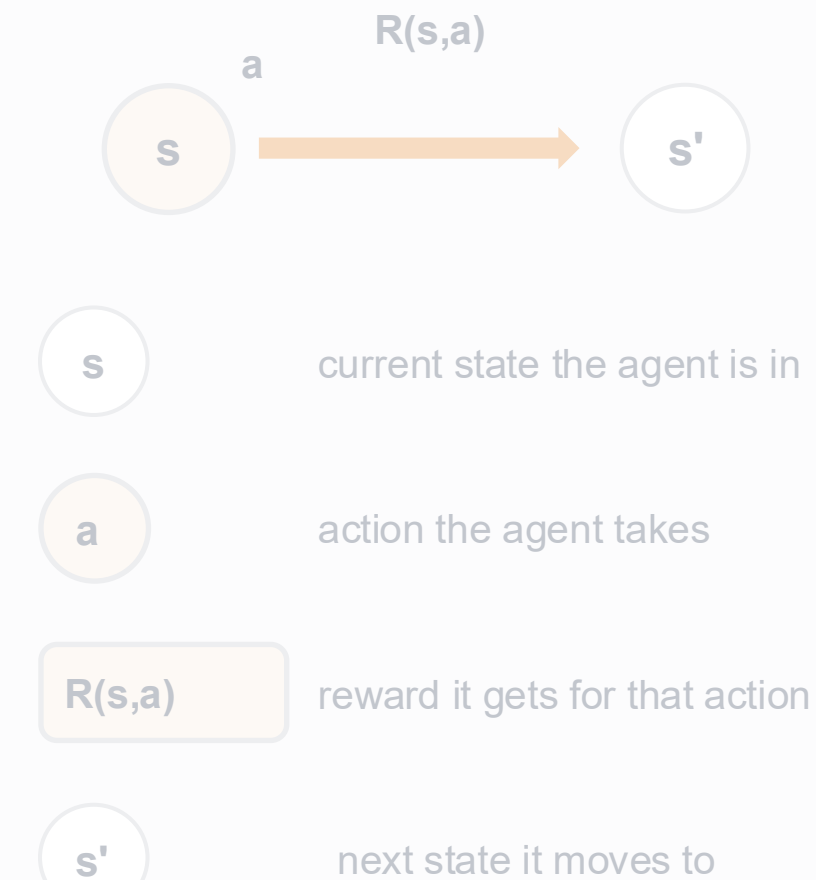
**1950s–70s
Foundations**
MDP, Bellman, DP

1980s–90s
Learning from Experience
TD and Q Learning

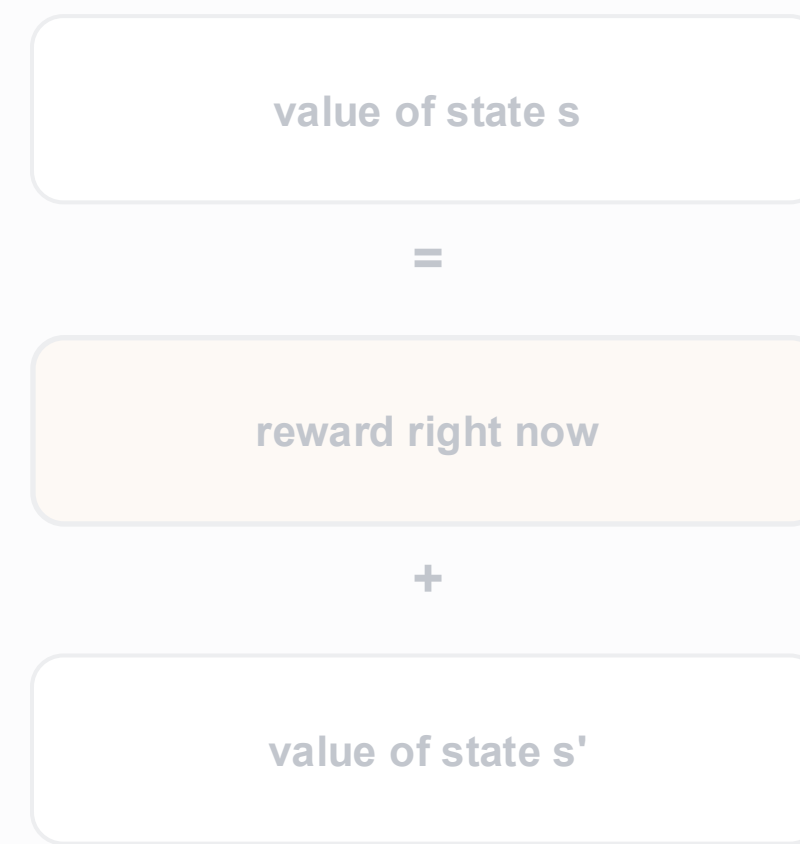
1990s–2015
Deep RL
Networks estimate values

2016–2026
Today & Beyond
RL in the real world

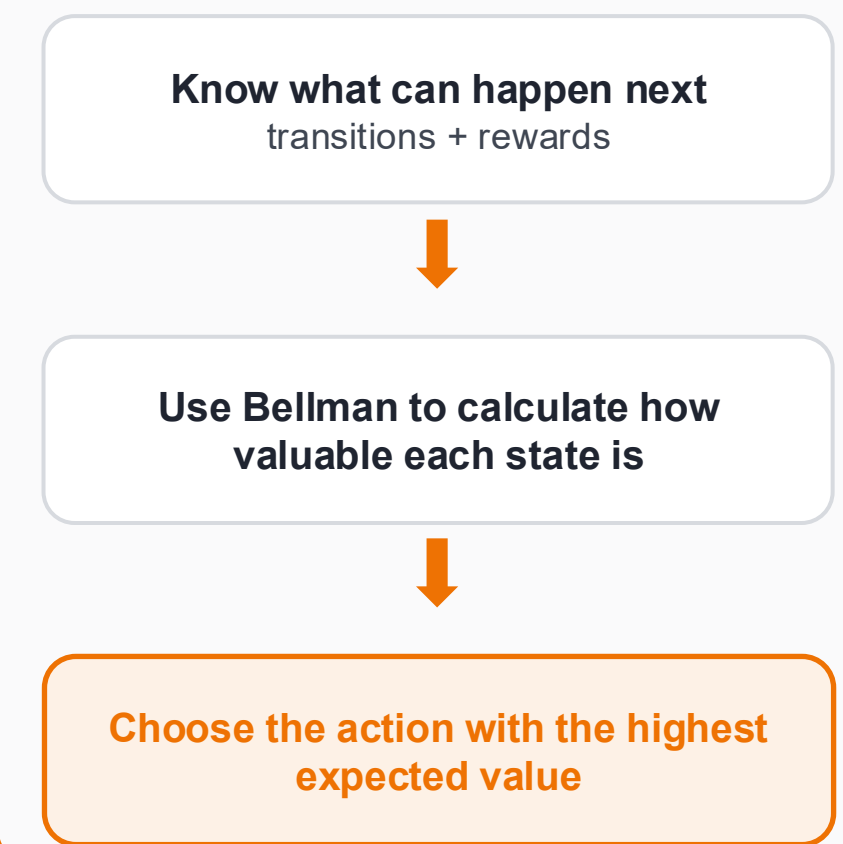
MDP: a mathematical description of the world as a decision problem



Bellman equation: links the value of today to reward now and future value



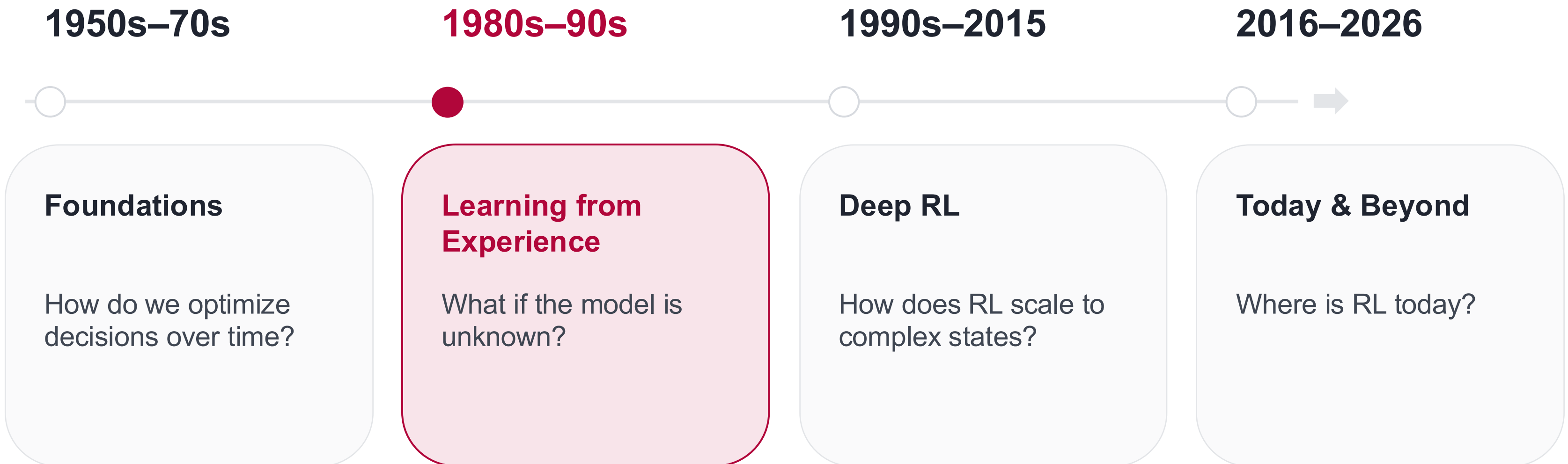
Dynamic Programming: computes the best action by sweeping all states, if the whole world is known



How Did We Get Here?

Four questions that moved reinforcement learning forward

kisz@hpi.de
hpi.de/kisz



1980s–90s

Can We Learn Without Knowing the Model?

TD Learning · Q-Learning

both learn from experienced rewards and transitions, and update their estimates step by step

TD Learning

learns how good a state is

$V(s)$: the value of a state

bootstrapping: updates an estimate using the next estimate

improved estimate of $V(s)$ after gaining enough experience

➔ **Q-Learning (1992)**

1950s–70s

Foundations

MDP, Bellman, DP

1980s–90s

**Learning from
Experience**

TD and Q Learning

1990s–2015

Deep RL

Networks estimate
values

2016–2026

Today & Beyond

RL in the real world

Q-Learning Formula

how a Q-value is updated from experience

$$Q'(s, a) = Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

new
Q-value

current
Q-value

new information

Formula symbols

$Q'(s,a)$	updated value	r	reward	s	state
$Q(s,a)$	current value estimate	γ	discount factor	a	action
α	learning rate	$\max Q(s',a')$	best next-state value	s'	next state

What learning parameters are there?

$$Q'(s, a) = Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

$$a = \begin{cases} \arg \max_{a'} Q(s, a') & \text{with probability } 1 - \epsilon, \\ \text{random action} & \text{with probability } \epsilon. \end{cases}$$



learning rate

determines how strongly new information influences the previous Q-value



discount factor

weights future rewards relative to immediate rewards



exploration rate

indicates how often random actions are chosen instead of the current best action

number of episodes

number of learning cycles the model goes through

initialization of Q-values

initial values of the Q-table

Example: Frozen Lake

Q-table initialized at 0, $\alpha = 0.1$, $\gamma = 0.95$, $r = +1$ at the goal, else 0

$$Q'(s, a) = Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{just } r \text{ if the episode ended}} - Q(s, a) \right]$$

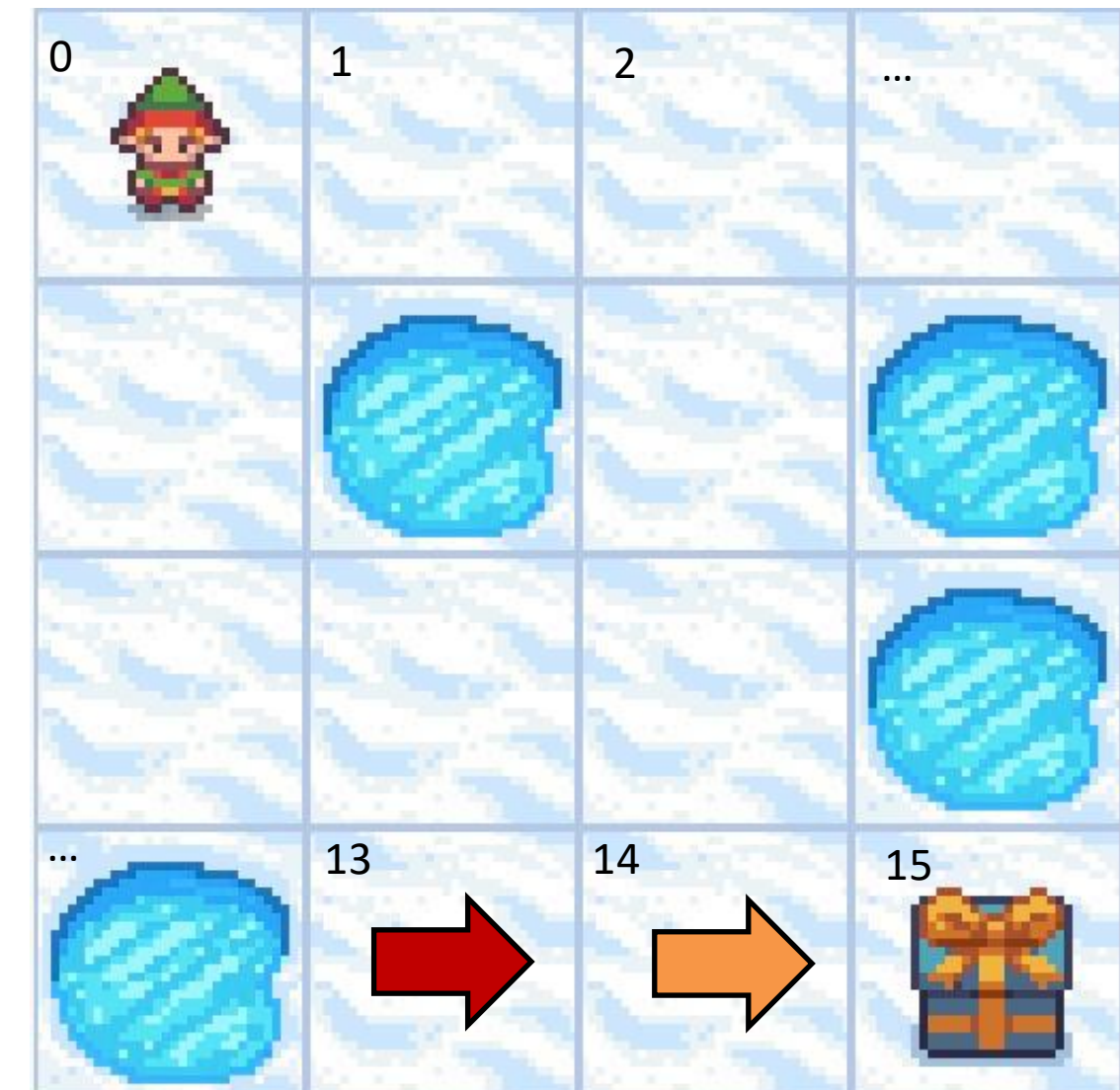
UPDATE 1 – reaching the goal by moving to the right in state 14

$$Q'(14, \rightarrow) = 0 + 0.1 * (1 - 0) = 0.1$$

UPDATE 2 – in the next episode, when visiting state 13 and moving to the right into state 14

$$Q'(13, \rightarrow) = 0 + 0.1 * (0 + 0.95 * 0.1 - 0) = 0.0095$$

...



RL Lab demo: Q-learning Training

Home

RL Lab

Interactive Reinforcement Learning Visualization

KI Service Zentrum
by Hasso-Plattner-Institut

HPI

Gefördert durch:
Bundesministerium
für Forschung, Technologie
und Raumfahrt

Configuration

Environment

FrozenLake-v1-NoSlip

Select environment

Algorithm

Q-Learning

Select algorithm

Random Seed

1

Same seed = same run. Click the dice to draw a new random seed.

START TRAINING

PLAY POLICY

EVALUATE POLICY

Learning Parameters

Num Episodes

500

Training episodes. Must be an integer.

Discount Factor

0.95

$0 \leq \gamma < 1$ - importance of future rewards

Environment

Ready

About FrozenLake (No Slip)

About Q-Learning

Policy: Q-Table

Min Q: 0.000

Max Q: 1.000

Avg Q: 0.309

0	0.45	0.50	0.68	0.00
0.49	0.77	0.81	0.11	0.42
0.09	0.00	0.86	0.00	0.00
0.33	0.00	0.69	0.00	0.00
0.00	0.00	0.90	0.00	0.00
0.00	0.00	0.64	0.00	0.00
0.00	0.04	0.18	0.95	0.00
0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.05	0.54	0.00
0.00	0.00	0.20	1.00	0.00
0.00	0.00	0.65	0.68	0.00
0.00	0.00	0.00	0.00	0.00

Lowest (global)

Highest (global)

Arrow colors show Q-values across the entire table (violet = global min, orange = global max). Cyan borders highlight the greedy action (highest Q-value) for each state (not shown if there's a tie). Grey tiles are terminal states (holes/goal): their values are never used or updated, so they keep whatever the initialization gave them and are excluded from the statistics.

Training Progress

Install RL Lab

Please follow the instructions on

<https://github.com/aihpi/workshop-rl1-introduction>

Quick start:

```
git clone https://github.com/aihpi/workshop-rl1-introduction.git  
cd workshop-rl1-introduction  
docker compose pull  
docker compose up -d
```

Then open **`http://localhost:3030`**

Ask us as soon as something fails, we try to get everyone running before the break.



Break

RL Lab demo: Slippery FrozenLake

Home

RL Lab

Interactive Reinforcement Learning Visualization

KI Service Zentrum
by Hasso-Plattner-Institut

HPI

Gefördert durch:
Bundesministerium
für Forschung, Technologie
und Raumfahrt

Configuration

Environment

FrozenLake-v1

Select environment

Algorithm

Q-Learning

Select algorithm

Random Seed

1

Same seed = same run. Click the dice to draw a new random seed.

START TRAINING

PLAY POLICY

EVALUATE POLICY

Learning Parameters

Num Episodes

5.0e+3

5000

Training episodes. Must be an integer.

Discount Factor

0.95

$0 \leq \gamma < 1$ - importance of future rewards

Environment

Ready

About FrozenLake (Slippery)

About Q-Learning

Policy: Q-Table

Min Q: 0.032

Max Q: 0.659

Avg Q: 0.220

0	0.14	0.13	0.12
0.15	0.14	0.13	0.10
0.14	0.10	0.12	0.09
0.17	0.12	0.03	0.00
0.13	0.00	0.09	0.00
0.24	0.24	0.23	0.00
0.17	0.29	0.28	0.00
0.18	0.33	0.23	0.00
0.00	0.30	0.63	0.00
0.00	0.35	0.57	0.00
0.00	0.46	0.64	0.00
0.00	0.39	0.66	0.00

Lowest (global)

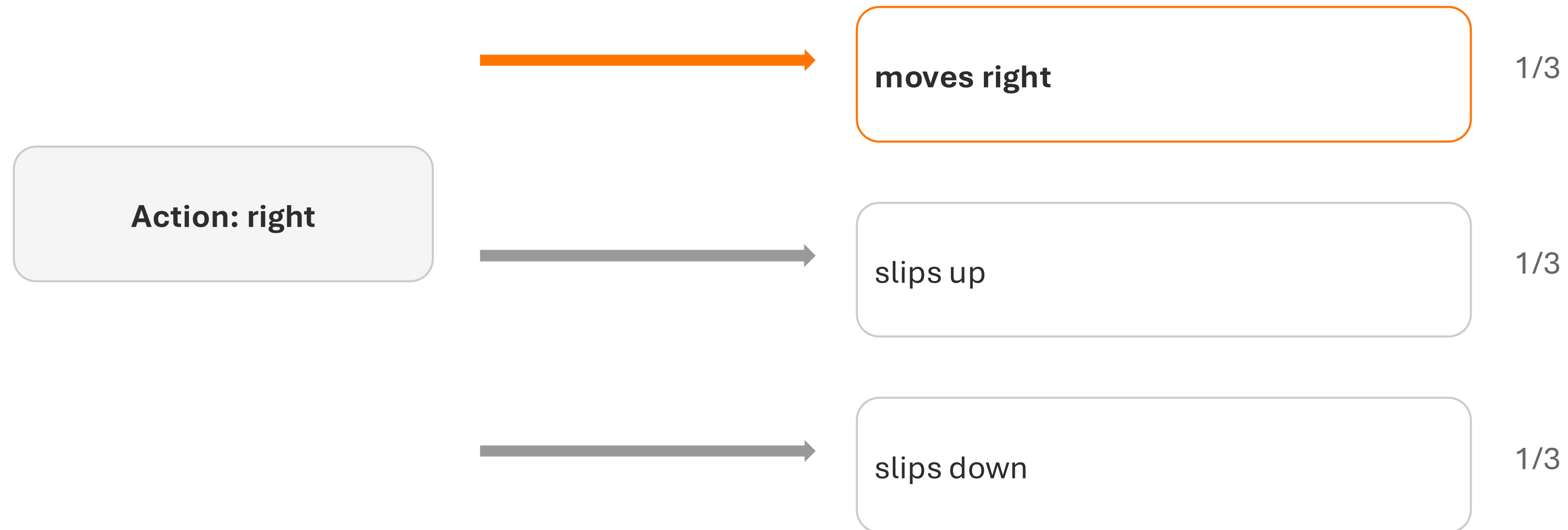
Highest (global)

Arrow colors show Q-values across the entire table (violet = global min, orange = global max). Cyan borders highlight the greedy action (highest Q-value) for each state (not shown if there's a tie). Grey tiles are terminal states (holes/goal): their values are never used or updated, so they keep whatever the initialization gave them and are excluded from the statistics.

Training Progress

When the ice is slippery

The same action no longer leads to the same next state.

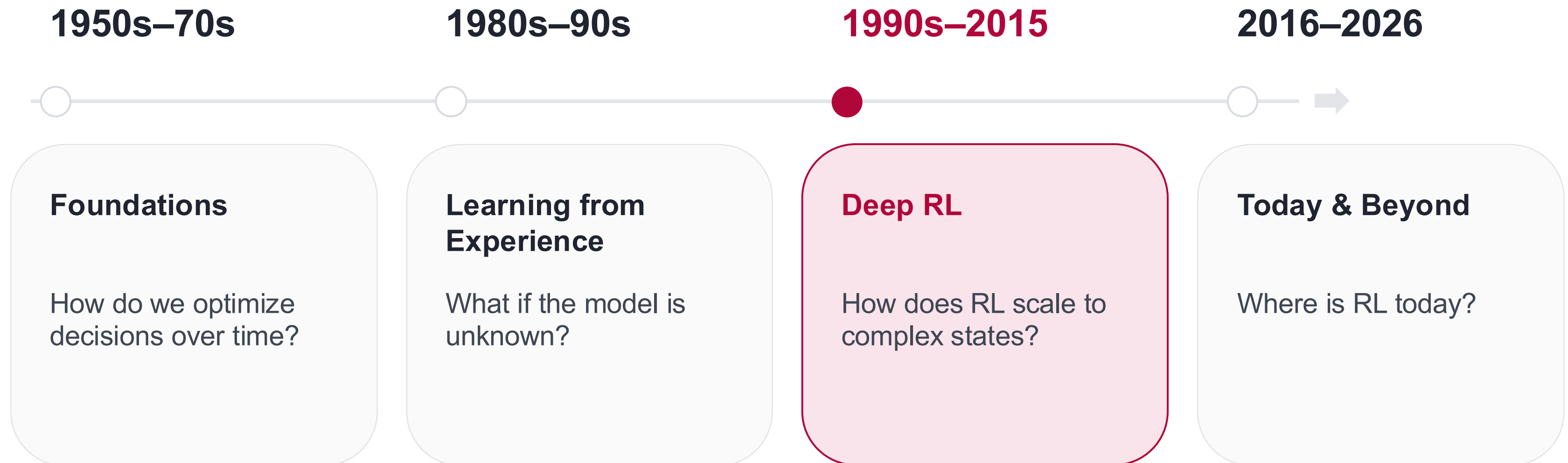


Learning takes longer, and the best policy can look counterintuitive.

How Did We Get Here?

Four questions that moved reinforcement learning forward

kisz@hpi.de
hpi.de/kisz



1990s–2015

How Does RL Scale to Complex States?

Deep RL · DQN · Function Approximation

1950s–70s

Foundations

MDP, Bellman, DP

1980s–90s

Learning from Experience

TD and Q Learning

1990s–2015

Deep RL

Networks estimate values

2016–2026

Today & Beyond

RL in the real world

Problem

a Q-table only works for small, discrete state spaces

fine for a handful of discrete states



Continuous state/action space



the table no longer fits

DQN

a deep neural network approximates the Q-function

the network predicts $Q(s,a)$ for any state



the RL idea stays exactly the same



the net replaces the table as function approximator

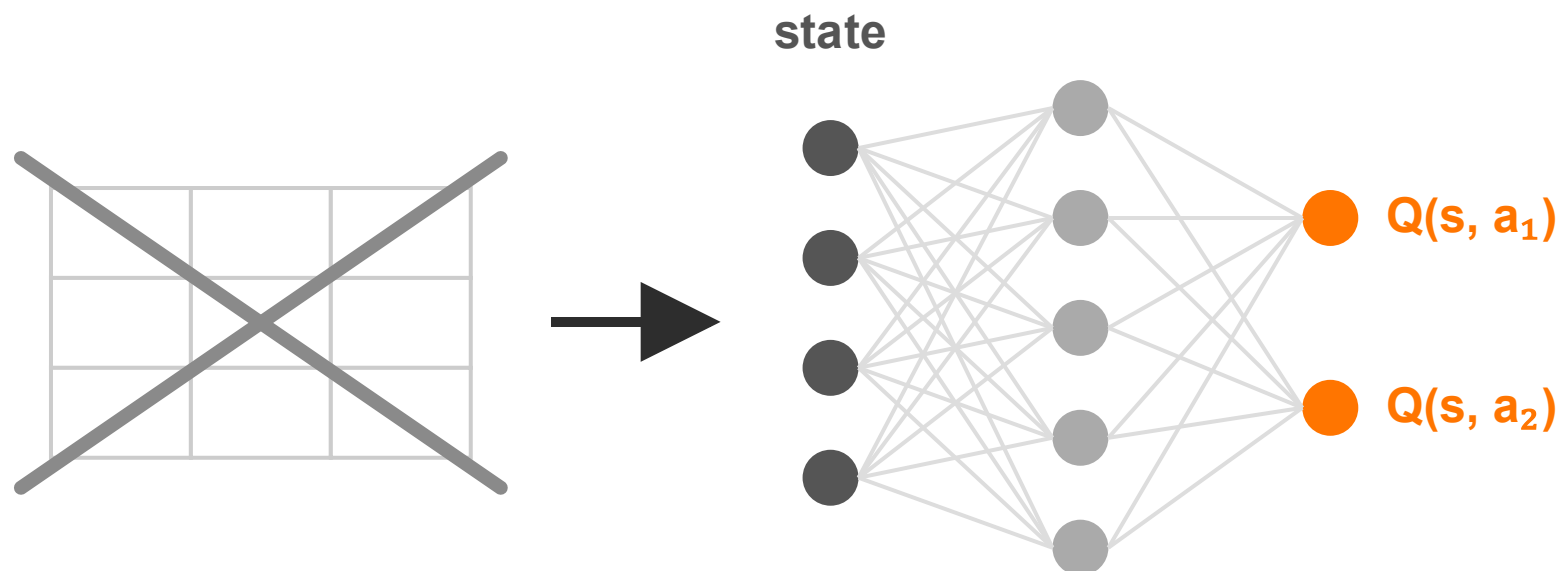
From Q-Learning to DQN

Same idea, same update — a network estimates $Q(s, a)$

CartPole: 4 continuous numbers

→ infinitely many states

→ **no table can hold them**



Same update rule, as a **gradient step**: $\text{loss} = (\text{target} - Q(s, a))^2$
backpropagation carries the error backwards through the layers —
every weight is nudged in proportion to its share of the error

TWO TRICKS MAKE IT STABLE

1 Replay Buffer

a memory of experiences (s, a, r, s') — train on random batches

→ one lucky success teaches the network thousands of times

2 Target Network

a frozen copy computes the targets $r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$

→ no chasing your own moving estimates

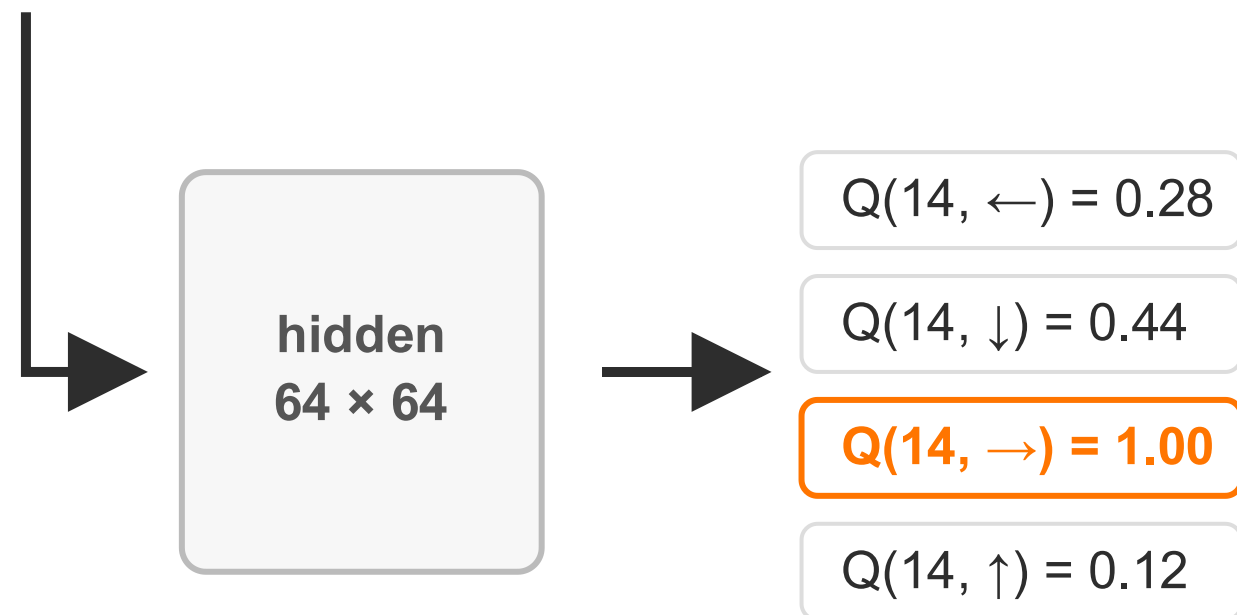
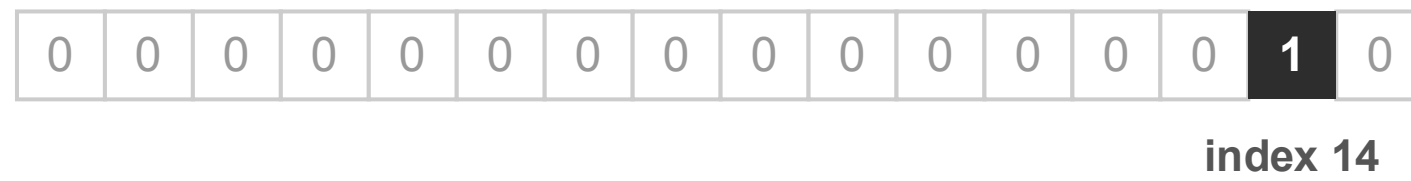
Shared weights: every update moves ALL states a little

DQN = Q-Learning + network + replay buffer + target network

DQN on FrozenLake

Only 16 states — ask the network all 16 questions, draw the answers as a table

state 14 as a **one-hot vector**:



All 16 states → **the familiar Q-table** (what RL Lab shows)

Q-Learning's table

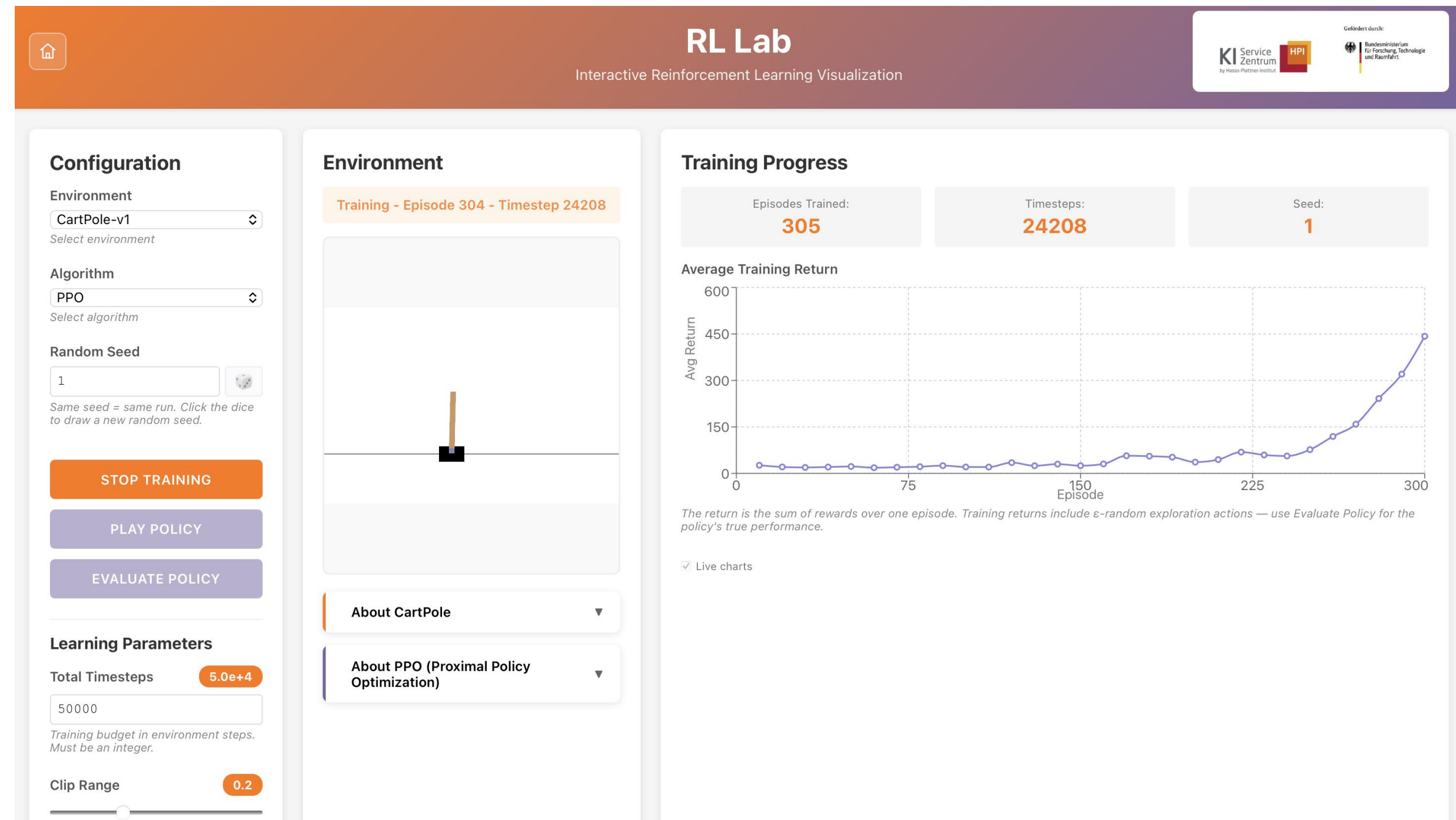
- every cell independent
- one update touches one cell
- values spread cell by cell

DQN's network

- all values share the weights
- every update moves ALL values
- terminal states: never trained, never used

RL Lab demo:

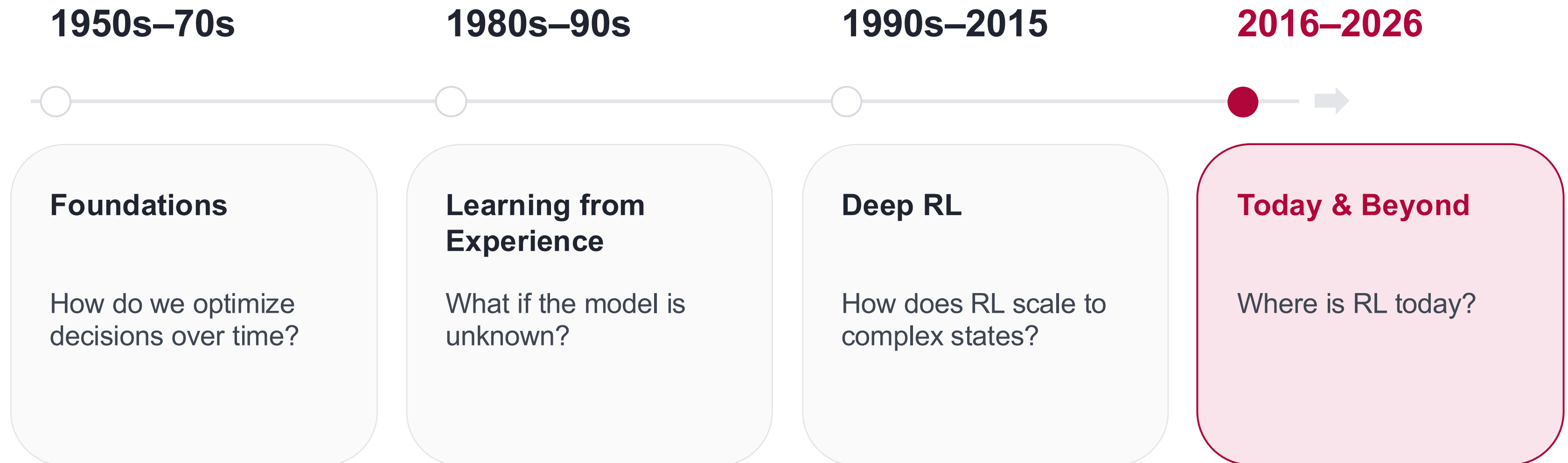
- DQN (alg)
- CartPole (env)
- PPO (alg)
- MountainCar (env)



How Did We Get Here?

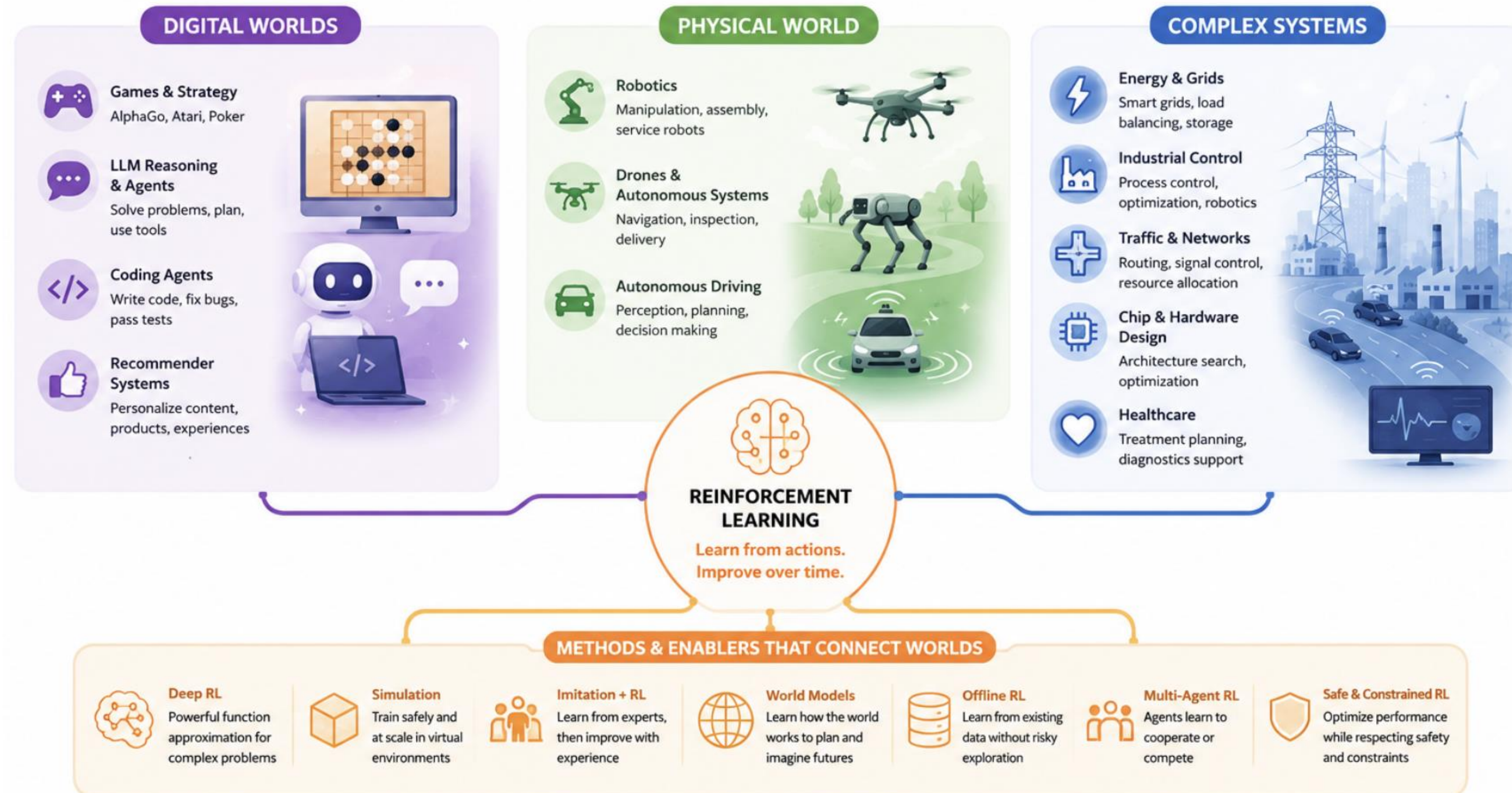
Four questions that moved reinforcement learning forward

kisz@hpi.de
hpi.de/kisz



The Scope of RL Nowadays

Where reinforcement learning is applied today



Autonomous Drone Racing

Can an AI learn to fly faster than a human?

Why RL?

- Continuous decisions at high speed
- Clear goal, but hard to program by hand

What does RL do?

Practise → **feedback** → **better policy**

Fast, clean laps are rewarded, crashes are not. Trained in simulation, then flown for real.

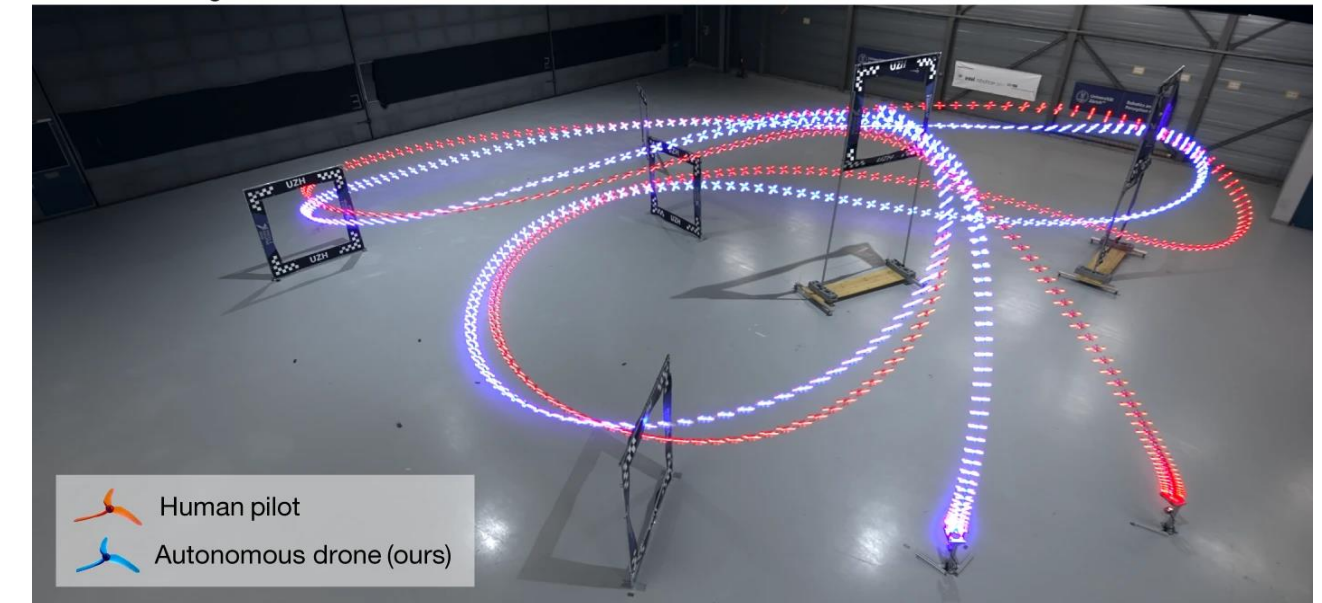
Why does it work?

- Millions of cheap, safe attempts
- Finds strategies humans would not code

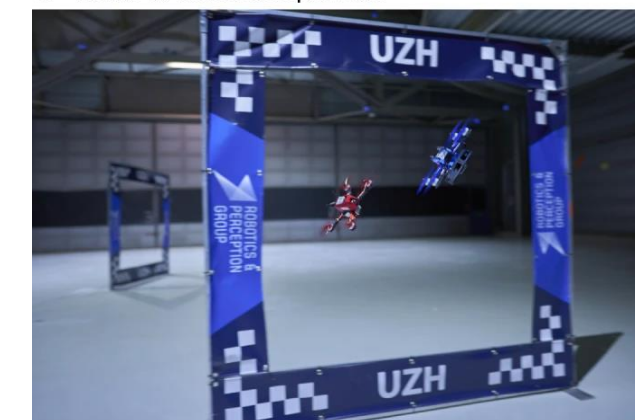
Take away

Swift beat the human world champions in direct races.

a Drone racing: human versus autonomous



b Head-to-head competition



c Human champions



Limitation

Sim-to-real gap limits transfer to reality. New tasks, environments or hardware often require adaptation or retraining.

RL for LLMs: Learning From Feedback

Can an AI learn which reasoning strategies actually work?

An LLM can produce many answers. Which behaviour should it learn to prefer?

RLHF: Human Feedback

Answers → **human preferences** → **model learns**

For goals with no correct answer: helpfulness, tone, preferred behaviour.

RLVR: Verifiable Rewards

Answer → **checked automatically** → **reward**

- Math: is the answer correct?
- Coding: do the tests pass?
- Agents: was the task completed?

Take away

RLHF: humans say what they prefer.

RLVR: the task says whether we succeeded.

Limitation

If success cannot be checked, a model may optimise the reward instead of the goal.

Challenges in Applying Reinforcement Learning

Why is it difficult to use RL in the real world?

Real experience is expensive

Learning can take thousands or millions of attempts, and a robot cannot crash a million times. Physical practice costs time, money and hardware.

Simulation is not reality

Sensors, friction, weather and people behave differently outside. A policy that works virtually can fail in reality: the sim-to-real gap.

Exploration can be dangerous

Trying out actions is fine in a game, but not in a factory, a car or a clinic. How can an agent explore without causing harm?

Rewards are hard to define

Real goals are more than plus one for success. Drive as fast as possible is not the same as drive safely and efficiently.

The more expensive, unpredictable and safety critical the environment, the harder RL becomes to apply.

When Should You Use Reinforcement Learning?

Conclusion: a short checklist before reaching for RL

RL is a good fit when ...

- ✓ **Decisions happen sequentially**
What you do now affects what happens next.
- ✓ **There is a measurable goal**
Success can be expressed through rewards or feedback.
- ✓ **The best action is not obvious**
Good behaviour must be discovered through experience.
- ✓ **Enough experience is available**
Through simulation, existing data or safe interaction.
- ✓ **Learning from consequences adds value**
A fixed rule, supervised model or classical optimisation is not enough.

Think twice when ...

- ✗ **Decisions are independent**
- ✗ **The correct action is already known**
- ✗ **Success cannot be evaluated**
- ✗ **Exploration is too expensive or dangerous**
- ✗ **A simpler optimisation approach solves the problem**

Interactive: Pen and Paper Exercise

Think of a problem where Reinforcement Learning could be useful.

Why does RL make sense, and how would you model it?



Define: Agent · Environment · State · Actions · Reward

Consider: Why RL instead of a simpler approach?



Outlook: What Is Coming in the Next Workshop

From today's concepts to a running implementation

kisz@hpi.de
hpi.de/kisz

Gymnasium API

Environments, observations, actions and rewards in code.

From problem to RL setup

One example problem, mapped step by step to agent, environment, state, actions and reward.

Algorithms: scope and choice

Which family fits which problem: value based or policy based, model free or model based, on or off policy, plus the theory behind them.

Addressing the challenges

How today's challenges are handled in practice, from exploration to reward design.

Outlook: Where to go from here

Notebooks, courses and references to keep going

Hands-on notebooks

Examples folder of the workshop repository

Stanford CS224R

cs224r.stanford.edu — full lecture series

OpenAI Spinning Up

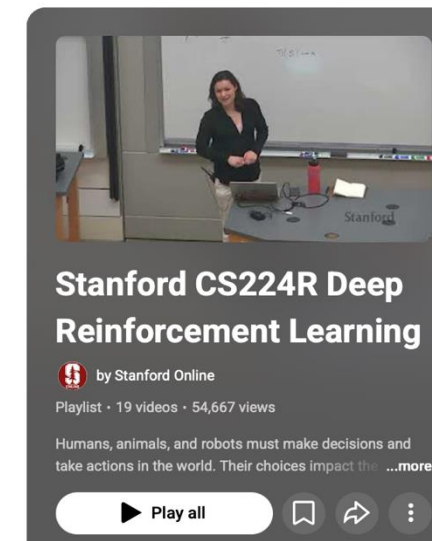
spinningup.openai.com/en/latest

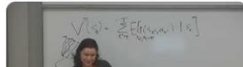
Stable-Baselines3 docs

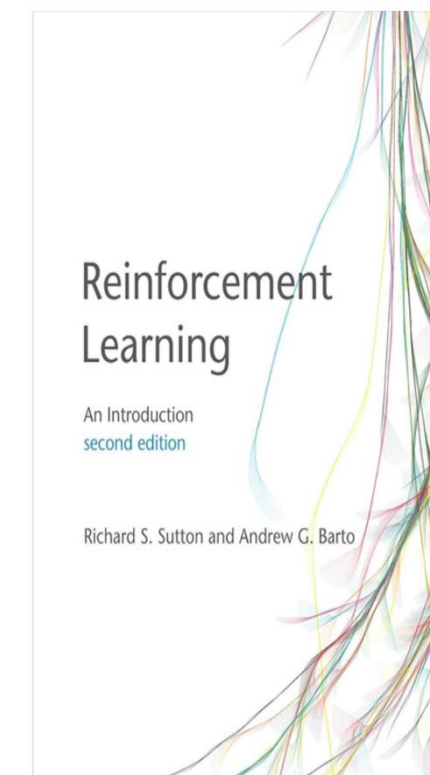
stablebaselines3.readthedocs.io/en/master

Sutton and Barto book

incompleteideas.net/book/RLbook2020.pdf



- 1  **Stanford CS224R Deep Reinforcement Learning | Spring 2025 | Lecture 1: Class Intro**
Stanford Online • 62K views • 2 months ago
- 2  **Stanford CS224R Deep Reinforcement Learning | Spring 2025 | Lecture 2: Imitation Learning**
Stanford Online • 13K views • 2 months ago
- 3  **Stanford CS224R Deep Reinforcement Learning | Spring 2025 | Lecture 3: Policy Gradients**
Stanford Online • 8.4K views • 2 months ago
- 4  **Stanford CS224R Deep Reinforcement Learning | Spring 2025 | Lecture 4: Actor-Critic Methods**
Stanford Online • 6.8K views • 2 months ago



kisz@hpi.de

hpi.de/kisz

Your opinion is
relevant!



QR code to feedback form



Gefördert durch:



Bundesministerium
für Forschung, Technologie
und Raumfahrt